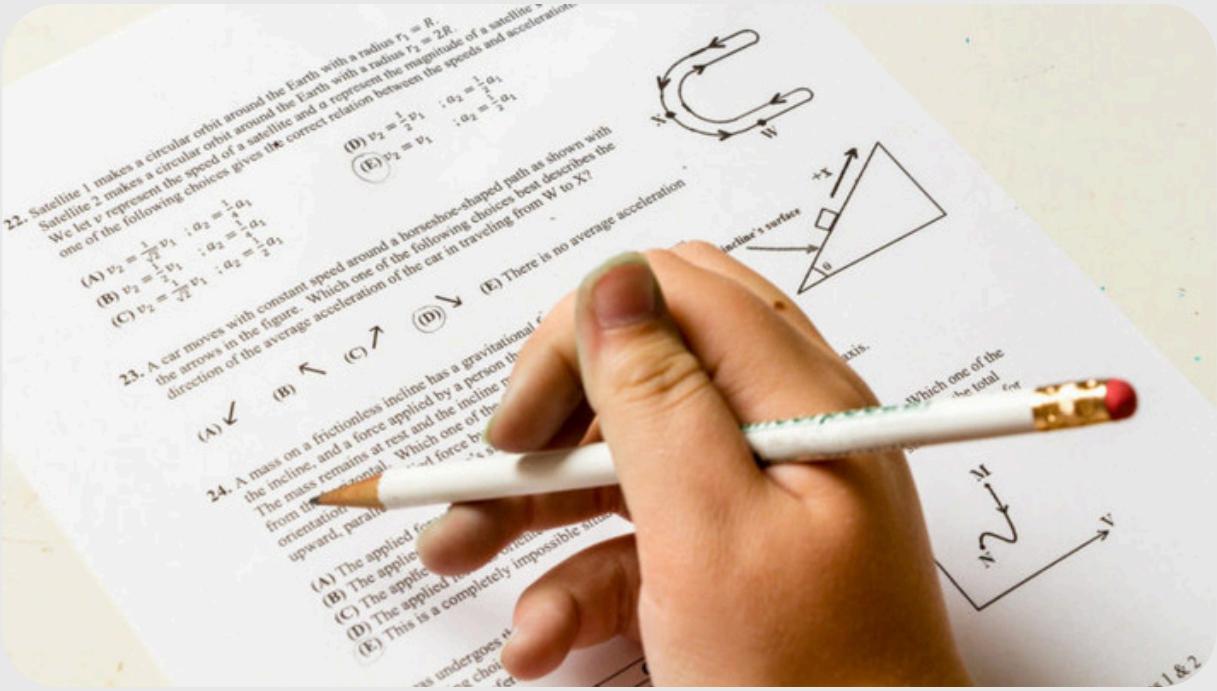
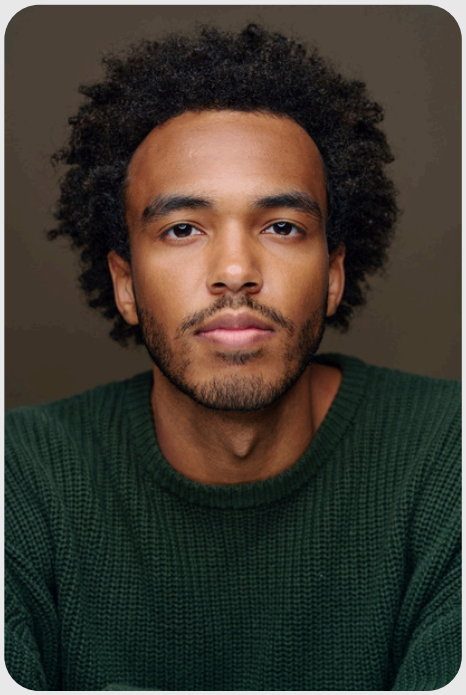
ASSESSMENT:

Are You Ready to Land a $100,000+ CodingJob in 2025?



BASHIRI SMITH

Hey!



A LITTLE ABOUT ME

My name is Bashiri Smith.

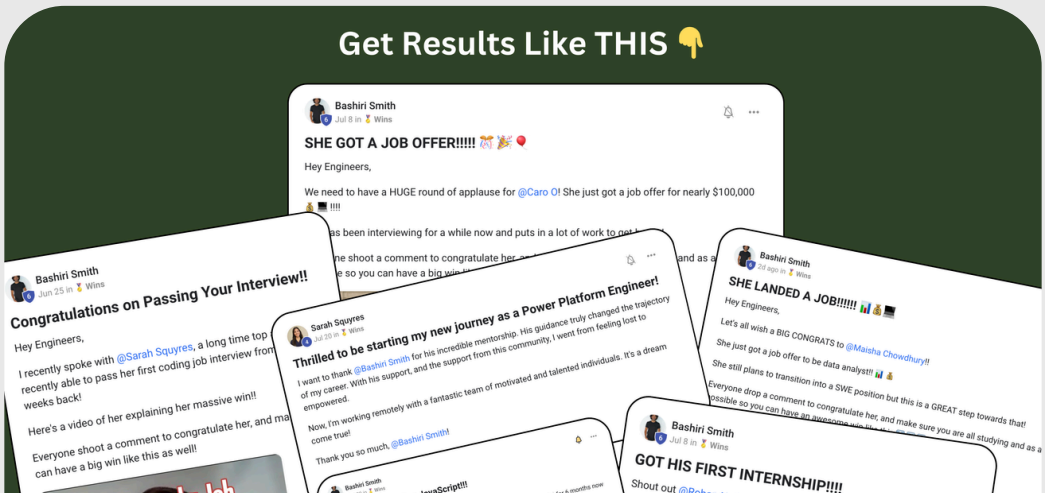
In just under 12 months & without a college degree or any work experience, I received 2 offers for more than $115,000 for my first job as a software engineer.

Since then, I have gone on to earn over $265,000p/y as a software engineer and I have also founded my own tech startup. (Interlade)

Throughout my journey to learn how to code, I've found that the unique information to ***ACTUALLY*** land a job is hard to come by regardless of if you go to college or a coding bootcamp.

My mission is to make it easy for anyone to change their life with coding!

This guide is for aspiring coders who want to earn $100k+ as a software engineer ASAP.



CONTENT

SECTION

01

Instructions

SECTION

02

Core Programming Concepts

SECTION

03

Building Basic Coding Projects

SECTION

04

Coding Comfort & Collaboration

SECTION

05

Scoring & Next Steps

01

Assessment Instructions

- This assessment is divided into three sections to evaluate your readiness to apply for entry-level software engineering positions.
- You may complete each section in any order, but be sure to provide detailed reasoning for your answers where applicable.
- Time yourself if you want to simulate a real interview or testing environment; aim to finish within 2–3 hours.
- Once finished, review your answers or share them with a mentor for feedback.

02

Core Programming Concepts

Goal: Confirm your understanding of fundamental programming concepts such as data types, control structures, functions, and complexity.

1. **Multiple Choice:**

a. Which of the following is not a primitive data type in many programming languages?

a. Integer
b. Boolean
c. String
d. Array

b. In the context of time complexity, what does Big-O notation represent?

a. The time it takes to compile your code
b. The upper bound of the growth rate of an algorithm
c. The amount of memory the program will use
d. The exact running time of an algorithm

2. **Short Answer:**

a. Explain the difference between a for loop and a while loop.
b. Describe how you would use a function (or method) to break your code into reusable pieces.

Core Programming Concepts

3. Code Snippet Explanation:
In your preferred programming language, consider the following pseudocode:

```plaintext
total = 0
for i in range(5):
    total = total + (i * 2)
print(total)
```

- Explain what the final printed value is and why.
- Discuss the time complexity of this code snippet.

03

Building Basic Coding Projects

Goal: Evaluate your ability to create small, functioning projects without relying on a step-by-step tutorial.

1. Coding Challenge — Console Application:
   - Task: Build a simple “To-Do List” console application in any language you prefer.
   - Requirements:
     i. The user can add tasks (e.g., “Buy groceries”).
     ii. The user can mark tasks as complete.
     iii. The application can display all tasks, along with their completion status.
   - Deliverables:
     iv. A short README describing how to compile and run the application.
     v. The source code demonstrating your approach.

Building Basic Coding Projects

2. Coding Challenge — Data Handling:
    - Task: Write a script that reads a small data file (JSON, CSV, or text) and outputs some form of processed result (e.g., summarizing sales data, counting word frequency, etc.).
    - Requirements:
        i. Parse the file to extract meaningful information.
        ii. Display the processed output in a user-friendly format.
    - Deliverables:
        iii. The script or source code.
        iv. A short explanation (2–3 sentences) about how your code processes the data and why you chose your approach.

Tip: Focus on clarity and simplicity. Employers often value code that is easy to read and maintain over code that is overly complex.

04

Coding Comfort & Collaboration

Goal: Assess how comfortable you are coding in front of others (or collaborating in real-time, such as in pair-programming sessions).

1. Pair-Programming Simulation (Self-Reflection):
   - Imagine you're asked to share your screen and walk through a simple coding task (like reversing a string or sorting a small list) with an interviewer watching.
   - Self-Reflection Prompt:
     i. What feelings or concerns come up for you when you think about coding in real-time with someone watching?
     ii. How might you address or reduce anxiety in these scenarios (e.g., prepare with practice problems, talk through each step out loud, etc.)?
2. Live Collaboration Exercise (Group Task):
   - If you have a friend or peer to practice with, try a 15-minute pair-programming session. If you don't have a friend you can find one in this free engineering group:

     https://www.skool.com/become-a-software-engineer

     - Pick a small problem (e.g., "Write a function that checks if a number is prime.").
     - Take turns writing code and explaining your thought process.
   - Reflection Prompt:
     - Note any communication barriers you encountered.
     - What went well in the collaborative process?

05 Scoring & Next Steps

There is no strict numerical score here—this assessment is qualitative. However, consider the following:

- Section A: Were your answers accurate and clear? Did you demonstrate core knowledge of data types, loops, functions, and complexity?
- Section B: Could you build at least one functional project without a tutorial? Did you document your approach effectively?
- Section C: Are you aware of your comfort level in pair-programming or live coding situations? Do you have a strategy for improving?

If you found that:

- Section A concepts were unclear, revisit a fundamentals course or resource.
- Section B tasks were difficult or incomplete, practice more small projects or coding challenges before moving forward.
- Section C caused undue stress, try more mock interviews or pair-programming sessions with friends to gain confidence.

By reviewing your performance in each area, you'll get a sense of whether you're ready to land that coding job in 2025.

Final Notes

- Practice is key. Focus on mastering the basics before attempting more advanced challenges.
- Review and refactor your work to keep learning. In interviews, being able to explain why you wrote code a certain way can be just as important as writing the code itself.
- Collaborate whenever possible. Real-world programming often involves working with others, so get comfortable discussing and reviewing code in a shared environment.

05

Still Need Help?

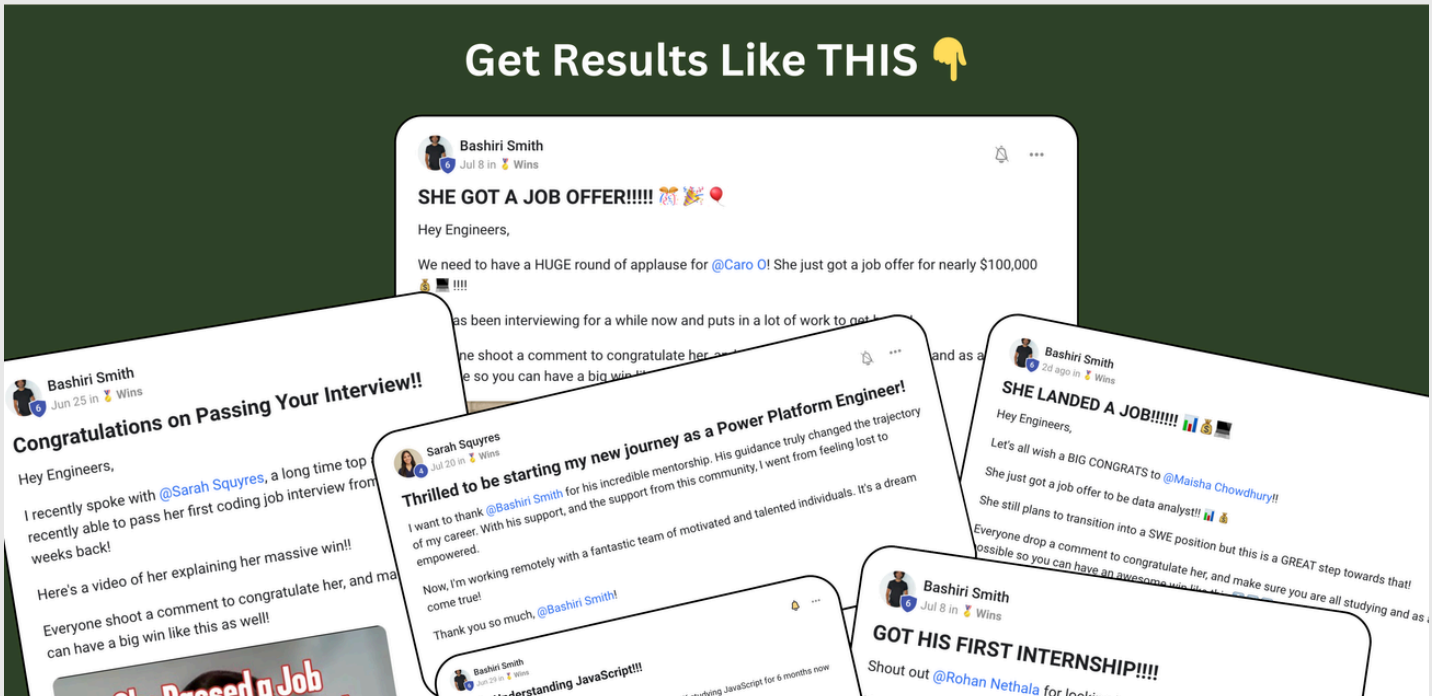
Learning how to code on your own can be extremely daunting and just make you feel so lost. I felt the same way while I was on my journey. So now I help aspiring coders who feel lost land their first $100,000+ coding job!

This mentorship is ONLY for serious people who want to land a job ASAP.

If this sounds like you we can set up a call!

On the call, you and I will come up with a custom plan of action personalized for you so you can see exactly how to become a $100,000+ software engineer.

This call is completely FREE and is guaranteed to help you get crystal clear on your next steps.

Fill out the form below and I will reach out to you ASAP!

https://api.leadconnectorhq.com/widget/form/8VbHZnjfk0r1SxbmHQ5r